

Software-Engineering-Best-Practices & Code-Modularisierung

Section 5 · Implementing CI-CD · Quelle: Kurs 4, M02 Kapitel 1–4

Databricks-Notebooks laden dazu ein, schnell loszulegen – doch genau das führt oft zu langen, monolithischen Skripten, die Ingestion, Transformation und Business-Logik in einer einzigen Zellenkette vermischen. Solcher Code lässt sich schwer testen, wiederverwenden oder von mehreren Personen gemeinsam pflegen. Software-Engineering-Best-Practices helfen dabei, aus Notebook-Prototypen wartbare, produktionsreife Data-Pipelines zu machen.

1. Kernprinzipien guter Data-Engineering-Praxis

- **Lesbarkeit:** klare Namen, konsistente Formatierung, kurze Funktionen – reduziert Fehler bei späteren Änderungen.
- **Dokumentation:** Docstrings und Kommentare, die das *Warum* statt das offensichtliche *Was* erklären.
- **Automatisiertes Testen:** Unit- und Integrationstests, die Regressionen frühzeitig aufdecken (siehe Kapitel 3 und 4 dieses Themenordners).
- **Versionskontrolle & Code-Review:** Änderungen nachvollziehbar machen und vor dem Merge gegenprüfen (siehe Kapitel 5).
- **Isolierte Umgebungen:** strikte Trennung von Entwicklungs-, Test- und Produktionsdaten (z. B. eigene Unity-Catalog-Kataloge je Umgebung).

2. Vom Monolithen zur modularen Funktion

Der zentrale Refactoring-Schritt ist, wiederkehrende Logik (Schema-Definitionen, Transformationen, Schreibvorgänge) aus dem Notebook-Ablauf in benannte, einzeln testbare Python-Funktionen auszulagern und in einem gemeinsamen Modul abzulegen. Aus dem folgenden Ausschnitt eines Bronze-Ingestion-Schritts ...

```
# Vorher: Schema-Definition direkt im Notebook-Flow
health_csv_schema = StructType([
    StructField("ID", IntegerType(), True),
    StructField("PII", StringType(), True),
    StructField("date", DateType(), True),
    StructField("HighCholest", IntegerType(), True),
    StructField("HighBP", DoubleType(), True),
    StructField("BMI", DoubleType(), True),
    StructField("Age", DoubleType(), True),
    StructField("Education", DoubleType(), True),
    StructField("income", IntegerType(), True)
])
```

... wird eine wiederverwendbare, dokumentierte Funktion in einem eigenen Modul `src/helpers/project_functions.py`:

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, DateType, DoubleType
from pyspark.sql.functions import when, col

def get_health_csv_schema():
    """Gibt das Schema für die Health-Data-CSV-Dateien zurück."""
    return StructType([
        StructField("ID", IntegerType(), True),
        StructField("PII", StringType(), True),
        StructField("date", DateType(), True),
        StructField("HighCholest", IntegerType(), True),
        StructField("HighBP", DoubleType(), True),
        StructField("BMI", DoubleType(), True),
        StructField("Age", DoubleType(), True),
        StructField("Education", DoubleType(), True),
        StructField("income", IntegerType(), True)
    ])

def high_cholest_map(col_name: str):
    """Wandelt einen numerischen Cholesterin-Code in ein lesbares Label um."""
    return (
        when(col(col_name) == 0, 'Normal')
        .when(col(col_name) == 1, 'Above Average')
        .when(col(col_name) == 2, 'High')
        .otherwise('Unknown')
    )
```

Im eigentlichen Notebook bleibt dann nur noch der eigentliche *Ablauf* übrig – lesbar als kurze Kette von Funktionsaufrufen:

```
from src.helpers.project_functions import get_health_csv_schema, high_cholest_map

health_csv_df = read_health_data(
    csv_path=f"/Volumes/{DA.catalog_name}/default/health",
    schema=get_health_csv_schema()
)
save_df_to_delta(health_csv_df, f"{DA.catalog_name}.default.health_bronze_dev",
mode="overwrite")
```

Der entscheidende Vorteil: Jede einzelne Funktion (Schema-Erzeugung, Mapping-Logik, Schreibvorgang) lässt sich isoliert mit Unit-Tests absichern, ohne dass dafür ein komplettes Notebook oder ein Spark-Cluster mit echten Produktionsdaten ausgeführt werden muss.

Ergänzt aus der offiziellen Databricks-Dokumentation: Databricks empfiehlt, Python-Code für Data-Engineering-Projekte nicht nur in Notebooks, sondern zusätzlich als reguläre **Workspace-Files** (`.py` -Module) im Git-Ordner abzulegen. Dadurch lässt sich derselbe Code sowohl per `import` aus Notebooks als auch direkt von einer lokalen IDE oder einer CI-Pipeline aus testen – eine Voraussetzung für die in Kapitel 3 behandelten Unit-Tests.